TimeTensors Experiment 1: Constant Filtering

1 Theoretical overview and goal

A constant look-back has zero empirical scale and carries little dynamic information. It can make normalized losses ill-conditioned and may correspond to missing-value imputation rather than a forecasting regime. Removing only constant windows changes the sample distribution; dropping every affected user also changes the population.

Experimental goal. Separate numerical/optimization benefits of excluding constant training windows from the easier evaluation obtained by excluding difficult test windows or entire users.

2 Implementation details

A look-back is flagged when it is constant or non-finite. Window removal and user removal are independently controlled for train and evaluation, yielding the keep, train-only, evaluation-only, and both-split comparisons. User removal drops a user if any accessible look-back in the selected split is flagged.

Raw dataset preparation first reads the sibling `config.json`. Portable top-level options, `timetensors`-scoped options, and run-specific additions share one precedence rule; curated `drop_users` lists are additive. These source exclusions precede, and are distinct from, the dynamic constant-window policies compared here.

The default study uses six datasets; $(168, 24)$, $(504, 168)$, and $(504, 504)$; DLinear; seeds 1–3; random sampling; batch size 256; learning rate 10^{-5}; 10,000 optimizer steps; and validation every 1,000 steps. It compares keep, drop-training-users, and drop-all-users first because prior experiments suggest that window removal adds little after affected users are dropped. The full profile retains all seven filtering policies. The launcher is `01_constants.slurm`.

Tables report a fixed test metric, seed mean $\pm$ sample standard deviation, and explicit per-row $\times 10^m$ scaling. The number of retained windows and users must accompany results: otherwise improvements caused by evaluating a smaller population are indistinguishable from model improvements.

3 Interpretation

Train-only filtering is the cleanest test of optimization benefit. Evaluation filtering and user dropping answer different population questions and should not be ranked directly against the unfiltered test without reporting coverage.